МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №7

з дисципліни **"Функціональне програмування"**

на тему **"FUNCTOOLS TA OPERATOR"**

Варіант 14. Логістичні відправлення

Виконала:
Студентка групи ФЕП-23
Поведа Анастасія

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: навчитися будувати функціональні конвеєри обробки даних за допомогою `functools.partial` і `functools.reduce`, замінювати lambda-гетери засобами модуля `operator`, створювати поліморфний API завантаження через `functools.singledispatch` і задавати повний порядок доменного класу через `functools.total_ordering` на прикладі рейтингу логістичних відправлень.

Теоретичні відомості:

Часткове застосування (`partial`) фіксує частину аргументів функції та повертає нову функцію з меншою кількістю параметрів. Завдяки цьому загальна функція з keyword-налаштуваннями, наприклад тарифами за кілограм і кілометр, перетворюється на готовий крок конвеєра без написання окремих обгорток. Згортка `reduce` послідовно пропускає акумулятор через елементи послідовності; початкове значення захищає від порожнього входу і фіксує тип результату. Якщо елементами послідовності є функції, `reduce` стає універсальним `pipeline(value, *funcs)`, який протягує значення через ланцюжок перетворень.

Модуль `operator` містить готові гетери: `itemgetter` для ключів словника, `attrgetter` для атрибутів об'єкта (з кількома іменами повертає кортеж, тому підходить для багатокритеріального сортування) та `methodcaller` для виклику методу з аргументами. `singledispatch` робить функцію генеричною за типом першого аргументу: окремі реалізації реєструються для `str`, `Path`, відкритого файлу чи списку, а виклик залишається єдиним. Декоратор `total_ordering` за `__eq__` та одним строгим порівнянням добудовує решту операторів; для несумісних типів методи мають повертати `NotImplemented`, щоб інтерпретатор міг спробувати дзеркальну операцію або коректно кинути `TypeError`.

Хід роботи:

1. У `core.py` реалізувала універсальний `pipeline` через `reduce` та обгортки `mapf` / `filt` як `partial(map, …)` і `partial(filter, …)`, щоб кожен крок конвеєра був

готовою функцією над ітерованим об'єктом:

```python
T = TypeVar("T")

Row = Mapping[str, Any]


def pipeline(value: T, *funcs: Callable[[Any], Any]) -> Any:
    """Протягує value через funcs зліва направо: pipeline(x, f, g) == g(f(x))."""
    return reduce(lambda acc, f: f(acc), funcs, value)


def mapf(fn: Callable[[Any], Any]) -> Callable[[Iterable[Any]], Iterable[Any]]:
    """partial-обгортка над map: mapf(fn)(iterable)."""
    return partial(map, fn)


def filt(pred: Callable[[Any], bool]) -> Callable[[Iterable[Any]], Iterable[Any]]:
    """partial-обгортка над filter: filt(pred)(iterable)."""
    return partial(filter, pred)
```

2. За варіантом 14 описала трансформації відправлень: `normalize_shipment` очищує ідентифікатор через `itemgetter` і `methodcaller("strip")` / `methodcaller("upper")` та приводить числові поля; `adjusted_cost` обчислює скориговану вартість з тарифів за вагу й відстань і надбавкою за пріоритет, а `has_priority` фільтрує за мінімальним пріоритетом — обидві мають keyword-параметри для `partial`; `with_adjusted` та `to_rank` повертають нові значення:

```python
get_id = itemgetter("id")
strip, upper = methodcaller("strip"), methodcaller("upper")


def normalize_shipment(row: Row) -> dict[str, Any]:
    """Новий запис із очищеним id та числовими полями."""
    return {
        "id": upper(strip(str(get_id(row)))),
        "weight": float(row["weight"]),
        "distance": float(row["distance"]),
        "priority": int(row["priority"]),
        "cost": float(row["cost"]),
    }


def adjusted_cost(
    row: Row,
    *,
    per_kg: float = 2.0,
    per_km: float = 0.5,
    priority_rate: float = 0.25,
) -> float:
    """Скоригована вартість: (cost + weight·per_kg + distance·per_km) × надбавка.

    Надбавка за пріоритет: 1 + priority_rate × (priority - 1).
    """
    base = row["cost"] + row["weight"] * per_kg + row["distance"] * per_km
    return float(base * (1 + priority_rate * (row["priority"] - 1)))

```

```python
def has_priority(row: Row, *, min_priority: int = 1) -> bool:
    """Предикат: пріоритет відправлення не нижче порога."""
    return bool(row["priority"] >= min_priority)


def with_adjusted(row: Row, pricer: Callable[[Row], float]) -> dict[str, Any]:
    """Новий запис із полем adjusted, обчисленим pricer."""
    return {**row, "adjusted": pricer(row)}


def to_rank(row: Row) -> ShipmentRank:
    """Перетворює словник на незмінний ShipmentRank."""
    return ShipmentRank(
        id=row["id"],
        weight=row["weight"],
        distance=row["distance"],
        priority=row["priority"],
        cost=row["cost"],
        adjusted=row["adjusted"],
    )
```

3. Функція `rank_shipments` збирає конвеєр: фільтр і калькулятор вартості створюються через `partial`, сортування — два стабільних проходи з `attrgetter("id")` та `attrgetter("priority", "adjusted")` (пріоритет спадно, вартість спадно, ідентифікатор зростом), потім Top-N. `total_cost` — агрегація `reduce(add, …)` над `attrgetter("adjusted")`:

```python
def rank_shipments(
    rows: Iterable[Row],
    *,
    top: int = 3,
    min_priority: int = 1,
    per_kg: float = 2.0,
    per_km: float = 0.5,
    priority_rate: float = 0.25,
) -> list[ShipmentRank]:
    """Конвеєр: нормалізація → фільтр за пріоритетом → вартість → сортування → Top-N."""
    accept = partial(has_priority, min_priority=min_priority)
    pricer = partial(
        adjusted_cost, per_kg=per_kg, per_km=per_km, priority_rate=priority_rate
    )
    result: list[ShipmentRank] = pipeline(
        rows,
        mapf(normalize_shipment),
        filt(accept),
        mapf(partial(with_adjusted, pricer=pricer)),
        mapf(to_rank),
        partial(sorted, key=attrgetter("id")),
        partial(sorted, key=attrgetter("priority", "adjusted"), reverse=True),
        lambda it: list(it)[:top],
    )
    return result


def total_cost(shipments: Iterable[ShipmentRank]) -> float:
    """reduce-агрегація: загальна скоригована вартість відправлень."""
    return reduce(add, map(attrgetter("adjusted"), shipments), 0.0)
```

4. Базовий приклад з методички (студенти) реалізувала тим самим інструментарієм: `clean_and_rank` нормалізує записи, відбирає повнолітніх і

сортує за score ↓, name ↑:

```python
get_name = itemgetter("name")
title = methodcaller("title")


def normalize_student(row: Row) -> dict[str, Any]:
    """Новий запис студента: name у Title Case, age → int, score → float."""
    return {
        "name": title(strip(str(get_name(row)))),
        "age": int(row["age"]),
        "score": float(row["score"]),
    }


def clean_and_rank(rows: Iterable[Row], top: int = 3) -> list[dict[str, Any]]:
    """Нормалізувати → повнолітні → score ↓, name ↑ → Top-N."""
    result: list[dict[str, Any]] = pipeline(
        rows,
        mapf(normalize_student),
        filt(lambda r: r["age"] >= 18),
        partial(sorted, key=itemgetter("name")),
        partial(sorted, key=itemgetter("score"), reverse=True),
        lambda it: list(it)[:top],
    )
    return result
```

5. У `models.py` описала незмінний клас `ShipmentRank` з `@total_ordering`: ключ порівняння `(priority, adjusted, id)`, власні `__eq__` / `__lt__` повертають `NotImplemented` для чужих типів, `__hash__` узгоджений з рівністю:

```python
@total_ordering
@dataclass(frozen=True, eq=False)
class ShipmentRank:
    """Відправлення в рейтингу; порядок: пріоритет ↑, дорожче краще, потім id."""

    id: str
    weight: float
    distance: float
    priority: int
    cost: float
    adjusted: float = 0.0

    def _key(self) -> tuple[int, float, str]:
        return (self.priority, self.adjusted, self.id)

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, ShipmentRank):
            return NotImplemented
        return self._key() == other._key()

    def __lt__(self, other: object) -> bool:
        if not isinstance(other, ShipmentRank):
            return NotImplemented
        return self._key() < other._key()

    def __hash__(self) -> int:
        return hash(self._key())
```

6. У `loaders.py` зробила поліморфний `load_shipments` на `singledispatch`: реалізації для `str` (JSON-рядок або шлях), `pathlib.Path` (JSON чи CSV за

розширенням), відкритого `io.TextIOBase` (формат за вмістом) і готового `list` з валідацією; базовий виклик кидає `TypeError` для непідтримуваних типів:

```python
def _check(rows: Any) -> Rows:
    """Мінімальна валідація готової колекції: список словників."""
    if not isinstance(rows, list) or any(not isinstance(r, Mapping) for r in rows):
        raise ValueError("Expected list of dicts")
    return [dict(r) for r in rows]


def _from_csv(text: str) -> Rows:
    """CSV-текст → список словників (значення лишаються рядками)."""
    return list(csv.DictReader(io.StringIO(text.strip())))


def _parse_text(text: str) -> Rows:
    """JSON, якщо текст починається з '[', інакше CSV."""
    stripped = text.strip()
    if stripped.startswith("["):
        return _check(json.loads(stripped))
    return _from_csv(text)


@singledispatch
def load_shipments(src: object) -> Rows:
    """Повертає список словників-відправлень із JSON/CSV-джерела будь-якого типу."""
    raise TypeError(f"Unsupported type: {type(src)!r}")


@load_shipments.register
def _(src: str) -> Rows:
    """JSON-рядок або шлях до .json/.csv."""
    if src.lstrip().startswith(("[", "{")):
        return _check(json.loads(src))
    return load_shipments(Path(src))


@load_shipments.register
def _(src: Path) -> Rows:
    """Файл .json або .csv за розширенням."""
    text = src.read_text(encoding="utf-8")
    if src.suffix.lower() == ".csv":
        return _from_csv(text)
    return _check(json.loads(text))


@load_shipments.register
def _(src: io.TextIOBase) -> Rows:
    """Відкритий текстовий файл: формат визначається за вмістом."""
    return _parse_text(src.read())


@load_shipments.register
def _(src: list) -> Rows:  # type: ignore[type-arg]
    """Готова колекція словників."""
    return _check(src)


load_students = load_shipments
"""Базовий приклад з методички: той самий loader для записів студентів."""
```

7. Увесь ввід-вивід зібрала у `main.py`: читання прикладів із JSON, CSV та відкритого файлу, друк рейтингу за мінімальним пріоритетом і розміром Top-N

з аргументів командного рядка, загальна вартість і демонстрація базового прикладу:



```python
def main(argv: list[str]) -> None:
    """Друкує рейтинг відправлень; аргументи: [min_priority top]."""
    min_priority = int(argv[0]) if argv else 2
    top = int(argv[1]) if len(argv) > 1 else 3

    rows_json = load_shipments(HERE / "sample_data.json")
    rows_csv = load_shipments(str(HERE / "sample_data.csv"))
    with open(HERE / "sample_data.json", encoding="utf-8") as f:
        rows_file = load_shipments(f)
    print(f"json={len(rows_json)} csv={len(rows_csv)} file={len(rows_file)} rows")

    ranked = rank_shipments(rows_json + rows_csv, top=top, min_priority=min_priority)
    for s in ranked:
        print(
            f"{s.id:<8} p={s.priority} {s.weight:>6.1f} kg {s.distance:>7.1f} km "
            f"base={s.cost:>7.2f} adjusted={s.adjusted:>8.2f}"
        )
    print(f"total adjusted cost: {total_cost(ranked):.2f}")

    students = load_students(
        '[{"name": " alice ", "age": "19", "score": "8.5"}, '
        '{"name": "bob", "age": "17", "score": "9.1"}]'
    )
    print("students:", clean_and_rank(students, top=2))


if __name__ == "__main__":
    main(sys.argv[1:])
```

8. Написала pytest-тести: `pipeline` застосовує функції зліва направо, `mapf`/`filt` є `partial`-об'єктами, `partial` над предикатом пріоритету й калькулятором вартості, сортування з `itemgetter` за кількома ключами, `methodcaller` та `attrgetter`:



```python
def test_pipeline_applies_left_to_right() -> None:
    assert pipeline(2, lambda x: x * 3, lambda x: x - 1, lambda x: x * x) == 25
    assert pipeline("same") == "same"


def test_mapf_filt_are_partials() -> None:
    assert isinstance(mapf(str), partial) and isinstance(filt(bool), partial)
    out = pipeline([1, 2, 3, 4, 5], filt(lambda x: x > 2), mapf(lambda x: x * 10))
    assert list(out) == [30, 40, 50]


def test_partial_priority_and_cost() -> None:
    urgent_only = partial(has_priority, min_priority=3)
    assert urgent_only({"priority": 3}) is True
    assert urgent_only({"priority": 2}) is False
    flat = partial(adjusted_cost, per_kg=0.0, per_km=0.0, priority_rate=0.0)
    assert flat({"cost": 50.0, "weight": 9.0, "distance": 9.0, "priority": 4}) == 50.0
    row = {"cost": 120.0, "weight": 12.5, "distance": 340.0, "priority": 2}
    assert adjusted_cost(row) == pytest.approx(393.75)


def test_operator_helpers() -> None:
    rows = [{"p": 2, "c": 5}, {"p": 2, "c": 1}, {"p": 1, "c": 9}]
    assert sorted(rows, key=itemgetter("p", "c")) == [
        {"p": 1, "c": 9},
        {"p": 2, "c": 1},
        {"p": 2, "c": 5},
    ]
    assert methodcaller("upper")(methodcaller("strip")("  sh-7 ")) == "SH-7"
    s = ShipmentRank("SH-1", 1.0, 1.0, 2, 10.0, 12.0)
    assert attrgetter("priority", "adjusted")(s) == (2, 12.0)
```

9. Тести конвеєра: нормалізація створює новий запис, Top-N у правильному порядку з очікуваними значеннями вартості, фільтрація за пріоритетом і тай-

брейкер за ідентифікатором, reduce-агрегація, відсутність мутацій вхідних даних:

```python
def test_normalize_creates_new_record(rows: list[dict[str, Any]]) -> None:
    original = deepcopy(rows)
    out = normalize_shipment(rows[0])
    assert out == {
        "id": "SH-101",
        "weight": 12.5,
        "distance": 340.0,
        "priority": 2,
        "cost": 120.0,
    }
    assert rows == original


def test_rank_shipments_top_n_and_order(rows: list[dict[str, Any]]) -> None:
    top = rank_shipments(rows, top=2, min_priority=2)
    assert [s.id for s in top] == ["SH-102", "SH-104"]
    assert [s.adjusted for s in top] == pytest.approx([1815.0, 975.0])
    everything = rank_shipments(rows, top=10)
    assert [s.id for s in everything] == [
        "SH-102",
        "SH-104",
        "SH-101",
        "SH-105",
        "SH-103",
    ]


def test_rank_shipments_filters_and_tiebreak() -> None:
    same = [
        {"id": "b", "weight": 1, "distance": 1, "priority": 1, "cost": 10},
        {"id": "a", "weight": 1, "distance": 1, "priority": 1, "cost": 10},
        {"id": "c", "weight": 1, "distance": 1, "priority": 1, "cost": 5},
    ]
    assert [s.id for s in rank_shipments(same, top=3)] == ["A", "B", "C"]
    assert rank_shipments(same, top=3, min_priority=2) == []


def test_total_cost_reduce(rows: list[dict[str, Any]]) -> None:
    top = rank_shipments(rows, top=3)
    assert total_cost(top) == pytest.approx(1815.0 + 975.0 + 393.75)
    assert total_cost([]) == 0.0


def test_no_mutation(rows: list[dict[str, Any]]) -> None:
    original = deepcopy(rows)
    rank_shipments(rows, top=5, min_priority=2)
    assert rows == original
```

10. Тести `singledispatch`: одні й ті самі дані з JSON-рядка, шляху `str`, `Path`, відкритого файлу (JSON і CSV) та списку дають однаковий результат;

непідтримуваний тип → `TypeError`, неправильна структура → `ValueError`:

```python
JSON = '[{"id": "x", "weight": 1, "distance": 1, "priority": 1, "cost": 1}]'
CSV = "id,weight,distance,priority,cost\nx,1,1,1,1\n"


def test_load_from_all_sources(tmp_path: Path) -> None:
    pj, pc = tmp_path / "s.json", tmp_path / "s.csv"
    pj.write_text(JSON, encoding="utf-8")
    pc.write_text(CSV, encoding="utf-8")
    expected = load_shipments(JSON)
    assert expected[0]["id"] == "x"
    assert load_shipments(str(pj)) == expected
    assert load_shipments(pj) == expected
    assert load_shipments(expected) == expected
    with open(pj, encoding="utf-8") as f:
        assert load_shipments(f) == expected
    with open(pc, encoding="utf-8") as f:
        assert load_shipments(f)[0]["id"] == "x"
    csv_rows = load_shipments(pc)
    assert csv_rows[0] == {
        "id": "x",
        "weight": "1",
        "distance": "1",
        "priority": "1",
        "cost": "1",
    }
    assert rank_shipments(csv_rows) == rank_shipments(expected)


def test_load_rejects_bad_input() -> None:
    with pytest.raises(TypeError):
        load_shipments(3.14)
    with pytest.raises(ValueError):
        load_shipments(["not", "dicts"])
    with pytest.raises(ValueError):
        load_shipments('{"id": "x"}')
```

11. Тести `total_ordering`: усі шість операторів, `sorted`/`max`, хешованість і `NotImplemented` для числа; базовий приклад зі студентами:

```python
def test_shipment_rank_ordering() -> None:
    a = ShipmentRank("a", 1.0, 1.0, 3, 10.0, adjusted=100.0)
    b = ShipmentRank("b", 1.0, 1.0, 3, 10.0, adjusted=50.0)
    c = ShipmentRank("c", 1.0, 1.0, 1, 10.0, adjusted=500.0)
    assert c < b < a
    assert a > b and a >= b and c <= b and a != b
    assert a == ShipmentRank("a", 9.0, 9.0, 3, 99.0, adjusted=100.0)
    assert sorted([a, c, b]) == [c, b, a] and max([a, b, c]) is a
    assert len({a, b, c, ShipmentRank("a", 9.0, 9.0, 3, 99.0, adjusted=100.0)}) == 3


def test_shipment_rank_not_implemented_for_other_types() -> None:
    s = ShipmentRank("a", 1.0, 1.0, 1, 10.0)
    assert s.__eq__(1) is NotImplemented
    assert s.__lt__(1) is NotImplemented
    assert s != 1
    with pytest.raises(TypeError):
        _ = s < 1


# --- базовий приклад з методички ---


def test_clean_and_rank_students() -> None:
    rows = load_students(
        '[{"name": "  ALICE  ", "age": "19", "score": "8.5"},'
        ' {"name": "bob", "age": "17", "score": "9.1"},'
        ' {"name": "Carol", "age": "21", "score": "7.0"},'
        ' {"name": "denis", "age": "20", "score": "9.3"}]'
    )
    assert clean_and_rank(rows, top=2) == [
        {"name": "Denis", "age": 20, "score": 9.3},
        {"name": "Alice", "age": 19, "score": 8.5},
    ]
```

12. Property-based тести на Hypothesis: Top-N завжди є префіксом повного відсортованого списку, конвеєр не мутує вхід і `total_cost` збігається з сумою вартостей, фільтр за мінімальним пріоритетом відбирає рівно потрібні записи:

```python
rows_st = st.lists(
    st.fixed_dictionaries(
        {
            "id": st.text(min_size=1, max_size=8),
            "weight": st.floats(min_value=0, max_value=1000, allow_nan=False),
            "distance": st.floats(min_value=0, max_value=5000, allow_nan=False),
            "priority": st.integers(min_value=1, max_value=4),
            "cost": st.floats(min_value=0, max_value=10000, allow_nan=False),
        }
    ),
    max_size=25,
)


@given(rows_st, st.integers(min_value=0, max_value=10))
def test_top_is_prefix_of_sorted(rows: list[dict[str, Any]], n: int) -> None:
    full = rank_shipments(rows, top=len(rows) + 1)
    assert rank_shipments(rows, top=n) == full[:n]
    assert full == sorted(full, key=lambda s: (-s.priority, -s.adjusted, s.id))


@given(rows_st)
def test_no_mutation_and_total(rows: list[dict[str, Any]]) -> None:
    original = deepcopy(rows)
    top = rank_shipments(rows, top=5, min_priority=2)
    assert rows == original
    assert abs(total_cost(top) - sum(s.adjusted for s in top)) < 1e-6


@given(rows_st, st.integers(min_value=1, max_value=4))
def test_filter_respects_min_priority(rows: list[dict[str, Any]], p: int) -> None:
    top = rank_shipments(rows, top=100, min_priority=p)
    assert all(isinstance(s, ShipmentRank) and s.priority >= p for s in top)
    assert len(top) == sum(1 for r in rows if r["priority"] >= p)
```

13. Запустила демо з параметрами за замовчуванням (пріоритет ≥ 2, Top-3) і з жорсткішими (пріоритет ≥ 3, Top-5): завантажено 4 + 3 + 4 записи з трьох джерел, відправлення відсортовано за пріоритетом і скоригованою вартістю:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab07> python main.py
json=4 csv=3 file=4 rows
SH-202   p=4  140.0 kg  2300.0 km base= 900.00 adjusted= 4077.50
SH-102   p=3   80.0 kg  1200.0 km base= 450.00 adjusted= 1815.00
SH-104   p=3   25.0 kg   600.0 km base= 300.00 adjusted=  975.00
total adjusted cost: 6867.50
students: [{'name': 'Alice', 'age': 19, 'score': 8.5}]

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab07> python main.py 3 5
json=4 csv=3 file=4 rows
SH-202   p=4  140.0 kg  2300.0 km base= 900.00 adjusted= 4077.50
SH-102   p=3   80.0 kg  1200.0 km base= 450.00 adjusted= 1815.00
SH-104   p=3   25.0 kg   600.0 km base= 300.00 adjusted=  975.00
total adjusted cost: 6867.50
students: [{'name': 'Alice', 'age': 19, 'score': 8.5}]
```

14. Перевірила проєкт автоматичними інструментами: усі 17 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають

зауважень до стилю:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab07> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs_functional_for_nastia
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 17 items

tests\test_lab07.py::test_pipeline_applies_left_to_right PASSED          [  5%]
tests\test_lab07.py::test_mapf_filt_are_partials PASSED                  [ 11%]
tests\test_lab07.py::test_partial_priority_and_cost PASSED               [ 17%]
tests\test_lab07.py::test_operator_helpers PASSED                        [ 23%]
tests\test_lab07.py::test_normalize_creates_new_record PASSED            [ 29%]
tests\test_lab07.py::test_rank_shipments_top_n_and_order PASSED          [ 35%]
tests\test_lab07.py::test_rank_shipments_filters_and_tiebreak PASSED     [ 41%]
tests\test_lab07.py::test_total_cost_reduce PASSED                       [ 47%]
tests\test_lab07.py::test_no_mutation PASSED                             [ 52%]
tests\test_lab07.py::test_load_from_all_sources PASSED                   [ 58%]
tests\test_lab07.py::test_load_rejects_bad_input PASSED                  [ 64%]
tests\test_lab07.py::test_shipment_rank_ordering PASSED                  [ 70%]
tests\test_lab07.py::test_shipment_rank_not_implemented_for_other_types PASSED [ 76%]
tests\test_lab07.py::test_clean_and_rank_students PASSED                 [ 82%]
tests\test_properties.py::test_top_is_prefix_of_sorted PASSED            [ 88%]
tests\test_properties.py::test_no_mutation_and_total PASSED              [ 94%]
tests\test_properties.py::test_filter_respects_min_priority PASSED       [100%]

============================== 17 passed in 0.90s ==============================

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab07> python -m mypy --strict .
Success: no issues found in 7 source files

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab07> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab07> python -m black --check .
All done! ✨ 🍰 ✨
7 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я побудувала функціональний конвеєр рейтингу логістичних відправлень (варіант 14) на `functools.partial`, `reduce` та гетерах `operator`: тарифи за вагу й відстань і поріг пріоритету фіксуються частковим застосуванням, сортування виконується за кількома ключами через `attrgetter`, а результат — Top-N незмінних об'єктів `ShipmentRank` із повним порядком від `total_ordering`; загальна вартість обчислюється згорткою `reduce`. Поліморфний `load_shipments` на `singledispatch` дав єдиний інтерфейс для JSON-рядка, шляху, відкритого файлу та готового списку. Юніт- і property-тести, mypy, ruff і black підтвердили коректність конвеєра, завантажувачів і всіх операторів порівняння.